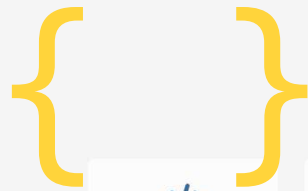


Python

{ Python Programming Introduction

Learn the fundamentals of Python programming



Syntax



Variables



Data Types



Numbers



Casting

Operators



Python Syntax Fundamentals

Indentation

- Indentation defines code blocks
- 4 spaces is the standard convention
- Incorrect indentation raises **IndentationError**

```
if True:
    print("Correct indentation")
```

Comments

- Single-line comments start with **#**
- Multi-line comments use triple quotes
- Comments are ignored by the interpreter

```
# This is a single-line comment
'''This is a multi-line comment.'''
```

Statements & Lines

- Statements typically end with a new line
- Use backslash for explicit line continuation
- Implicit continuation with parentheses/brackets

```
total = 1 + 2 + \
3 + 4

numbers = [1, 2, 3,
4, 5, 6]
```

Best Practices

- Avoid multiple statements on one line
- Use meaningful names for variables/functions
- Follow PEP 8 style guide

Variables and Naming Rules

Variable Declaration

- Variables are created when assigned a value
- No explicit declaration required
- Values determine variable type

```
age = 30           # Integer
name = "Alice"     # String
is_student = True  # Boolean
```

Dynamic Typing

- Python infers type from assigned value
- Same variable can hold different types
- Types checked at runtime

```
x = 42             # x is an integer
x = "Hello"        # x is now a string
x = False          # x is now a boolean
```

Naming Rules

- Start with letter or underscore
- Contain letters, numbers, underscores
- Case-sensitive (age ≠ Age)
- No Python keywords

Valid Names:

my_var _private var123 CamelCase snake_case

Built-in Data Types Overview

Text Type

str

Represents textual data enclosed in quotes.

```
text = "Hello, Python!"  
type(text) # <class 'str'>
```

Numeric Types

int

float

complex

Numbers, including integers, floats, and complex numbers.

```
integer = 42  
float = 3.14  
complex = 2 + 3j  
type(integer) # <class 'int'>
```

Sequence Types

list

tuple

range

Ordered collections that can be indexed and iterated.

```
list_example = [1, 2, 3]  
tuple_example = (1, 2, 3)  
type(list_example) # <class 'list'>
```

Mapping Type

dict

Unordered collections of key-value pairs.

```
person = {"name": "Alice", "age": 30}  
type(person) # <class 'dict'>
```

Boolean Type

True

False

Represents truth values, result of logical comparisons.

```
is_active = True  
type(is_active) # <class 'bool'>
```

Set Types

set

Unordered collections of unique elements.

```
unique_numbers = {1, 2, 3, 3}  
type(unique_numbers) # <class 'set'>
```

Numeric Types in Detail

Integers (int)

- Represent whole numbers without fractions
- Unlimited precision (limited by memory)
- Supports arbitrary precision arithmetic

```
my_integer = 100
print(type(my_integer)) # <class 'int'>
```

Floats (float)

- Represent real numbers with a decimal point
- Can be specified using scientific notation
- Typically double precision in Python

```
my_float = 20.5
scientific = 2.5e2 # 2.5 * 10^2 = 250.0
print(type(my_float)) # <class 'float'>
```

Numeric Operations Example

```
int_a = 10
int_b = 3
# Integer division
print(int_a // int_b) # 3
float_a = 10.0
float_b = 3.0
# Float division
print(float_a / float_b) # 3.3333333333333335
```

Type Casting and Conversion

int()

Converts to integer (truncates decimal part)

```
float_num = 10.7
int_from_float = int(float_num)
# Result: 10 (decimal part truncated)
```

```
str_int = "45"
int_from_str = int(str_int)
# Result: 45
```

float()

Converts to floating-point number

```
int_num = 5
float_from_int = float(int_num)
# Result: 5.0
```

```
str_float = "67.89"
float_from_str = float(str_float)
# Result: 67.89
```

A str()

Converts to string representation

```
num_to_string = 123
string_from_int = str(num_to_string)
# Result: "123"
```

```
bool_val = True
str_from_bool = str(bool_val)
# Result: "True"
```

⚠ Error Handling

Converting non-numeric strings raises **ValueError**:

```
try:
    invalid_str_int = int("hello")
except ValueError as e:
    print(e)
```

>_ Output:

invalid literal for int() with base 10: 'hello'

Arithmetic Operators

+ Addition

+ Adds two operands

Example: $5 + 2 = 7$

- Subtraction

- Subtracts second operand from first

Example: $4 - 2 = 2$

× Multiplication

* Multiplies two operands

Example: $2 * 3 = 6$

÷ Division

/ Divides first operand by second

Example: $4 / 2 = 2.0$

⌊ Floor Division

// Divides and floors result

Example: $10 // 3 = 3$

⌘ Modulo

% Returns remainder of division

Example: $5 \% 2 = 1$

</> Code Example

```
# Arithmetic Operators Example
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulo:", a % b)
print("Exponentiation:", a ** b)
```

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulo: 1
Exponentiation: 1000
```

Assignment and Comparison Operators

= Assignment Operators

Operators used to assign values to variables. Can be used with other operators for shorthand operations.

Operator	Example	Meaning
=	a = 7	Basic assignment
+=	a += 1	a = a + 1
-=	a -= 3	a = a - 3
*=	a *= 4	a = a * 4

```
x = 5
print("Initial x:", x)
x += 3 # Equivalent to x = x + 3
print("x after x += 3:", x)
x -= 2 # Equivalent to x = x - 2
```

≠ Comparison Operators

Operators that compare two values and return a Boolean result (**True** or **False**).

Operator	Meaning	Example	Result
==	Equal to	3 == 5	False
!=	Not equal to	3 != 5	True
>	Greater than	3 > 5	False
<	Less than	3 < 5	True

```
a = 7
b = 5
print("a == b is", a == b)
print("a != b is", a != b)
print("a > b is", a > b)
print("a < b is", a < b)
```

Logical Operators



Boolean Logic Operations

and

Both conditions must be True

or

At least one condition must be True

not

Reverses the boolean value



Truth Tables

AND Operator

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False

OR Operator

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



Code Examples

and Operator

```
x = True
y = False
print(x and y) # Output: False
```

or Operator

```
x = True
y = False
print(x or y) # Output: True
```



Best Practices

- Use parentheses to clarify complex expressions
- Short-circuit evaluation: **and** stops at first False

```
# Clear and readable
if (age >= 18 and has_license) or can_drive:
    print("Eligible to drive")
```